

Student 10 **Gudi Yashwanth** 192524275RD

2 / 2 submitted

Q1. Student Grade Book Application

CODE

```
# Student Grade Book Application

def grade_from_total(total):
    if total ≥ 90:
        return "O"
    elif total ≥ 80:
        return "A+"
    elif total ≥ 70:
        return "A"
    elif total ≥ 60:
        return "B+"
    elif total ≥ 50:
        return "B"
    else:
        return "F"

def main():
    # (a) Dictionary: key = name, value = list of 5 subject marks
    students = {
        "Asha": [85, 90, 78, 88, 92],
        "Karthik": [65, 70, 60, 58, 75],
        "Meera": [92, 88, 95, 85, 90],
        "Vignesh": [55, 48, 52, 60, 49],
        "Divya": [72, 68, 75, 70, 80],
    }

    # (b) Compute total, average, and grade for each student
    results = {
        name: {
            "marks": marks,
            "total": sum(marks),
            "avg": sum(marks) / len(marks),
            "grade": grade_from_total(sum(marks))
        }
        for name, marks in students.items()
    }

    # (c) Class topper (highest average) and subject with highest overall mean
    topper = max(results.items(), key=lambda x: x[1]["avg"])[0]

    # Highest overall mean subject index
    subject_means = []
    num_subjects = 5
    for i in range(num_subjects):
        mean_i = sum(results[name]["marks"][i] for name in results) / len(results)
        subject_means.append(mean_i)

    highest_subject_index = max(range(num_subjects), key=lambda i: subject_means[i])
    subject_names = ["Subject1", "Subject2", "Subject3", "Subject4", "Subject5"]
    highest_subject = subject_names[highest_subject_index]

    # (d) Display formatted result table sorted by average (descending)
    sorted_names = sorted(results.keys(), key=lambda name: results[name]["avg"], reverse=True)

    print("STUDENT GRADE BOOK RESULTS")
    print("-" * 85)
    print(f"{'Name':<12}{'S1':>6}{'S2':>6}{'S3':>6}{'S4':>6}{'S5':>6}"
          f"{'Total':>8}{'Avg':>8}{'Grade':>8}")
    print("-" * 85)

    for name in sorted_names:
        marks = results[name]["marks"]
        total = results[name]["total"]
        avg = results[name]["avg"]
        grade = results[name]["grade"]

        print(f"{'name':<12}{'marks[0]':>6}{'marks[1]':>6}{'marks[2]':>6}{'marks[3]':>6}{'marks[4]':>6}"
```


SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report
f"{total:>8}{avg:>8.2f}{grade:>8}")

```
print("-" * 85)
print(f"Class Topper (Highest Average): {topper} (Avg: {results[topper]['avg']:.2f})")
print(f"Subject with Highest Overall Mean: {highest_subject} (Mean: {subject_means[highest_subject_index]:.2f})")

if __name__ == "__main__":
    main()
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))
def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] >= amount:
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(('Withdraw', amount))
        else:
            print('Insufficient balance')
    else:
        print('Account does not exist')
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

OUTPUT

(no output)